UNIVERSIDADE PAULISTA – UNIP

E-BOOK EDUCATIVO DE EXTENSÃO

PROGRAMAÇÃO PARA INICIANTES

Conceitos, lógica e primeiros programas em Python

Curso: Sistemas de Informação

Material educativo para a comunidade

SÃO PAULO

2026

APRESENTAÇÃO

A programação está presente em grande parte das tecnologias utilizadas atualmente. Aplicativos de celular, sites, sistemas empresariais, jogos, plataformas digitais e diversos serviços dependem de programas para funcionar.

Apesar de muitas pessoas associarem programação a conceitos complexos, seus fundamentos podem ser aprendidos gradualmente. O primeiro passo é compreender como um problema pode ser dividido em etapas e transformado em instruções que um computador consiga executar.

Este e-book foi desenvolvido para apresentar os fundamentos da programação de maneira simples, prática e acessível. O conteúdo começa com conceitos de lógica e algoritmos e avança para variáveis, operadores, estruturas condicionais, repetições, funções e estruturas de dados.

A linguagem Python é utilizada como ferramenta prática ao longo do material. Sua sintaxe relativamente simples permite que o estudante visualize os conceitos apresentados e desenvolva pequenos programas desde os primeiros capítulos.

Além da teoria, o e-book apresenta exemplos de código, atividades de fixação e um projeto prático de controle de gastos. O objetivo é incentivar o aprendizado pela prática e mostrar como a programação pode ser utilizada para solucionar problemas do cotidiano.

O material não pretende formar um programador profissional em poucas páginas. Seu objetivo é oferecer uma base inicial para que o leitor compreenda os principais conceitos e tenha condições de continuar seus estudos de maneira autônoma.

OBJETIVO

Oferecer uma base inicial para compreender lógica, Python e os principais conceitos de programação.

SUMÁRIO

1. INTRODUÇÃO À PROGRAMAÇÃO 1

2. ALGORITMOS E LÓGICA DE PROGRAMAÇÃO 2

3. PRIMEIRO PROGRAMA EM PYTHON 3

4. VARIÁVEIS E TIPOS DE DADOS 4

5. OPERADORES 5

6. ENTRADA E SAÍDA DE DADOS 6

7. ESTRUTURAS CONDICIONAIS 7

8. ESTRUTURAS DE REPETIÇÃO 8

9. FUNÇÕES E ORGANIZAÇÃO DO CÓDIGO 9

10. LISTAS E ESTRUTURAS DE DADOS 10

11. ERROS E DEBUGGING 11

12. BOAS PRÁTICAS DE PROGRAMAÇÃO 12

13. PROJETO PRÁTICO: CONTROLE DE GASTOS 13

14. EXERCÍCIOS DE FIXAÇÃO 14

15. CHECKLIST PARA QUEM ESTÁ COMEÇANDO 15

16. CONSIDERAÇÕES FINAIS 16

17. REFERÊNCIAS E GLOSSÁRIO 17

1. INTRODUÇÃO À PROGRAMAÇÃO

Programação é o processo de criar instruções para que um computador realize tarefas.

Quando uma pessoa utiliza um aplicativo para pedir comida, realiza uma compra pela internet, joga um jogo ou acessa uma rede social, existe um conjunto de programas trabalhando para executar essas ações.

Programas são construídos para solucionar problemas e automatizar tarefas.

Por exemplo, imagine uma empresa que precisa calcular o valor total das compras de seus clientes. Em vez de realizar os cálculos manualmente, um programa pode receber os valores, processá-los e apresentar automaticamente o resultado.

Programar significa transformar uma necessidade em uma sequência de instruções que possa ser executada por um computador.

POR QUE APRENDER PROGRAMAÇÃO

- raciocínio lógico
- capacidade de resolver problemas
- organização
- criatividade
- pensamento analítico
- autonomia tecnológica

VOCÊ SABIA?

Programação não é apenas escrever código. Antes de escrever as instruções, é necessário entender o problema que precisa ser resolvido.

ATIVIDADE

Pense em uma tarefa do cotidiano que poderia ser automatizada por um programa. Exemplos: controlar gastos; organizar tarefas; calcular notas; controlar estoque; registrar clientes.

2. ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

Antes de escrever código, é importante saber organizar a solução de um problema.

Um algoritmo é uma sequência organizada de passos utilizada para resolver um problema ou realizar uma tarefa.

Uma receita culinária pode ser considerada um exemplo de algoritmo.

EXEMPLO

- Para preparar um sanduíche:
- 1. pegar o pão;
- 2. adicionar o recheio;
- 3. adicionar os demais ingredientes;
- 4. fechar o sanduíche;
- 5. servir.

PSEUDOCÓDIGO

- Problema: calcular a média de duas notas.
- 1. receber a primeira nota;
- 2. receber a segunda nota;
- 3. somar as duas notas;
- 4. dividir o resultado por 2;
- 5. apresentar a média.

DECOMPOSIÇÃO DE PROBLEMAS

- cadastro de clientes
- cadastro de produtos
- controle de estoque
- registro de vendas
- emissão de relatórios

ATIVIDADE

Escreva os passos necessários para criar um programa que calcule a média de três notas.

3. PRIMEIRO PROGRAMA EM PYTHON

Python é uma linguagem de programação utilizada em diversas áreas, como desenvolvimento web, automação, análise de dados e inteligência artificial.

Sua sintaxe é considerada relativamente simples, o que pode facilitar o aprendizado dos primeiros conceitos.

PRIMEIRO EXEMPLO

print("Olá, mundo!")

OUTRO EXEMPLO

PYTHON

```python
print("Bem-vindo ao estudo de programação!")
print("Este é meu primeiro programa.")
```

COMENTÁRIOS

PYTHON

```python
# Este programa apresenta uma mensagem
print("Olá!")
```

DICA

Não é necessário memorizar todos os comandos. O mais importante no início é entender o que cada parte do programa faz.

ATIVIDADE

Crie um programa que apresente seu nome, sua idade e seu curso.

4. VARIÁVEIS E TIPOS DE DADOS

Variáveis são utilizadas para armazenar informações durante a execução de um programa.

EXEMPLO

```python
nome = "Maria"
idade = 20
altura = 1.65
```

PRINCIPAIS TIPOS

- String: textos. Ex.: nome = "João"
- Integer: números inteiros. Ex.: idade = 23
- Float: números decimais. Ex.: preco = 49.90
- Boolean: valores verdadeiros ou falsos. Ex.: aprovado = True

ALTERANDO VALORES

```python
idade = 20
idade = 21
```

ATIVIDADE

Crie variáveis para armazenar nome, idade, cidade, preço de um produto e se uma pessoa está cadastrada.

5. OPERADORES

Operadores permitem realizar operações com valores.

OPERADORES ARITMÉTICOS

Operador	Significado
+	Soma
-	Subtração
*	Multiplicação
/	Divisão

EXEMPLO

PYTHON

```python
preco = 50
quantidade = 3
total = preco * quantidade
print(total)
```

COMPARAÇAO

PYTHON

```python
idade = 20
print(idade >= 18)
```

OPERADORES DE COMPARAÇAO

- > maior que
- < menor que
- >= maior ou igual
- <= menor ou igual
- == igual
- != diferente

OPERADORES LÓGICOS

PYTHON

```python
idade = 20
cadastrado = True
print(idade >= 18 and cadastrado)
```

ATIVIDADE

Crie um programa que receba dois números e apresente soma, subtração, multiplicação e divisão.

6. ENTRADA E SAÍDA DE DADOS

Um programa pode receber informações do usuário e apresentar resultados.

Em Python, podemos utilizar input() para receber dados.

EXEMPLO

PYTHON

```python
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

RECEBENDO NUMEROS

PYTHON

```python
idade = int(input("Digite sua idade: "))
print(idade)
```

NUMEROS DECIMAIS

preco = float(input("Digite o preço: "))

EXEMPLO PRÁTICO

PYTHON

```python
nome = input("Nome: ")
idade = int(input("Idade: "))
print("Nome:", nome)
print("Idade:", idade)
```

ATIVIDADE

Crie um programa que pergunte nome, idade e cidade. Depois, apresente todas as informações na tela.

7. ESTRUTURAS CONDICIONAIS

Programas frequentemente precisam tomar decisões.

IF

```python
idade = 20
if idade >= 18:
print("Maior de idade")
```

IF E ELSE

```python
idade = 16
if idade >= 18:
print("Maior de idade")
else:
print("Menor de idade")
```

ELIF

```python
nota = 7
if nota >= 7:
print("Aprovado")
elif nota >= 5:
print("Recuperação")
else:
print("Reprovado")
```

CONDIÇOES COMBINADAS

```python
idade = 25
cadastrado = True
if idade >= 18 and cadastrado:
print("Acesso permitido")
```

ATIVIDADE

Crie um programa que receba uma nota e informe: Aprovado (≥ 7); Recuperação (5 a 6,9); Reprovado (< 5).

8. ESTRUTURAS DE REPETIÇÃO

Algumas tarefas precisam ser executadas várias vezes. Em vez de escrever o mesmo comando repetidamente, podemos utilizar estruturas de repetição.

FOR

PYTHON

```python
for numero in range(1, 11):
print(numero)
```

WHILE

PYTHON

```python
contador = 1
while contador <= 5:
print(contador)
contador = contador + 1
```

CUIDADO COM REPETIÇÕES INFINITAS

É importante garantir que a condição de parada seja alcançada. Um erro na lógica pode fazer o programa continuar executando indefinidamente.

EXEMPLO PRÁTICO

PYTHON

```python
for numero in range(1, 6):
resultado = numero ** 2
print(resultado)
```

ATIVIDADE

Crie um programa que apresente a tabuada de um número informado pelo usuário.

9. FUNÇÕES E ORGANIZAÇÃO DO CÓDIGO

Funções são blocos de código criados para realizar tarefas específicas. Elas ajudam a organizar programas e evitar repetição.

FUNÇÃO SIMPLES

```python
def saudacao():
print("Olá! Seja bem-vindo.")
saudacao()
```

FUNÇAO COM PARAMETRO

```python
def saudacao(nome):
print("Olá," nome)
saudacao("Maria")
```

FUNÇAO COM RETORNO

```python
def somar(a, b):
return a + b
resultado = somar(10, 5)
print(resultado)
```

POR QUE UTILIZAR FUNÇOES

- organizar o código
- evitar repetição
- facilitar manutenção
- dividir problemas
- reutilizar funcionalidades

ATIVIDADE

Crie uma função chamada calcular_media() que receba três notas e retorne a média.

10. LISTAS E ESTRUTURAS DE DADOS

Programas frequentemente precisam armazenar vários valores. Uma lista pode guardar diversos elementos.

CRIANDO UMA LISTA

produtos = ["Mouse", "Teclado", "Monitor"]

ACESSANDO ELEMENTOS

print(produtos[0])

PERCORRENDO UMA LISTA

PYTHON

```python
for produto in produtos:
print(produto)
```

ADICIONANDO ELEMENTOS

produtos.append("Fone")

REMOVENDO ELEMENTOS

produtos.remove("Mouse")

OUTRAS ESTRUTURAS

- tuplas
- conjuntos
- dicionários

DICIONÁRIO

PYTHON

```python
cliente = {
"nome": "João",
"idade": 25,
"cidade": "São Paulo"
}
print(cliente["nome"])
```

ATIVIDADE

Crie uma lista com cinco produtos e utilize um for para apresentar todos eles.

11. ERROS E DEBUGGING

Erros fazem parte do desenvolvimento de software. Até programadores experientes encontram problemas em seus códigos.

O importante é saber identificar e corrigir esses problemas.

ERRO DE SINTAXE

PYTHON

```python
if idade >= 18
print("Maior")
```

ERRO DE LOGICA

media = (nota1 + nota2) / 3

DEBUGGING

- 1. ler a mensagem de erro
- 2. identificar a linha indicada
- 3. verificar os valores utilizados
- 4. testar partes menores
- 5. corrigir
- 6. executar novamente

IMPORTANTE

Um erro não significa que você não sabe programar. Encontrar e corrigir erros faz parte do desenvolvimento de software.

12. BOAS PRÁTICAS DE PROGRAMAÇÃO

Além de fazer o programa funcionar, é importante escrever código organizado.

USE NOMES CLAROS

- Evite: x = 50
- Prefira: preco_produto = 50

EVITE REPETIÇÃO

Se uma mesma lógica aparece muitas vezes, considere transformá-la em uma função.

ORGANIZE O CÓDIGO

Separe as funcionalidades para facilitar a manutenção.

COMENTÁRIOS

```python
# Calcula o valor total da compra
total = preco * quantidade
```

TESTE SEU PROGRAMA

- 18
- 0
- número negativo
- texto

CONTROLE DE VERSÃO

Em projetos maiores, ferramentas de controle de versão permitem acompanhar alterações no código e facilitar o trabalho em equipe. Um exemplo bastante utilizado é o Git.

13. PROJETO PRÁTICO: CONTROLE DE GASTOS

Nesta etapa, os conceitos apresentados serão reunidos em um pequeno projeto.

O objetivo é criar um programa simples capaz de registrar gastos e calcular o valor total.

ETAPA 1 — CRIAR A LISTA

despesas = []

ETAPA 2 — RECEBER UM VALOR

```python
valor = float(input("Digite o valor da despesa: "))
despesas.append(valor)
```

ETAPA 3 — CALCULAR O TOTAL

```python
total = sum(despesas)
print("Total gasto:", total)
```

PROJETO COMPLETO

```python
despesas = []
quantidade = int(input("Quantas despesas deseja cadastrar? "))
for i in range(quantidade):
valor = float(input("Digite o valor da despesa: "))
despesas.append(valor)
total = sum(despesas)
print("Total gasto: R$", total)
```

COMO O PROGRAMA FUNCIONA

- 1. Cria uma lista vazia.
- 2. Pergunta quantas despesas serão cadastradas.
- 3. Utiliza uma repetição.
- 4. Recebe cada valor.
- 5. Armazena os valores na lista.
- 6. Soma todas as despesas.
- 7. Apresenta o resultado.

DESAFIO

- descrição da despesa
- categoria
- maior gasto
- menor gasto
- média das despesas

14. EXERCÍCIOS DE FIXAÇÃO

Os exercícios abaixo servem para praticar os fundamentos apresentados nos capítulos anteriores.

EXERCÍCIO 1 — VARIÁVEIS

Crie variáveis para armazenar nome, idade, profissão e salário. Apresente todas na tela.

EXERCÍCIO 2 — CÁLCULO

Crie um programa que receba o preço de um produto e sua quantidade. Calcule o valor total: total = preço × quantidade.

EXERCÍCIO 3 — CONDIÇÃO

Crie um programa que receba a idade de uma pessoa e informe: Criança para idade inferior a 12; Adolescente para idade entre 12 e 17; Adulto para idade igual ou superior a 18.

EXERCÍCIO 4 — REPETIÇÃO

Crie um programa que apresente os números de 1 a 20 utilizando for.

EXERCÍCIO 5 — FUNÇÃO

Crie uma função que receba dois números e retorne o maior deles.

EXERCÍCIO 6 — LISTA

Crie uma lista contendo cinco nomes. Utilize um for para apresentar cada nome.

EXERCÍCIO 7 — PROJETO

Crie um pequeno programa de cadastro de produtos. O programa deve: 1. solicitar o nome; 2. solicitar o preço; 3. armazenar as informações; 4. apresentar os produtos cadastrados.

15. CHECKLIST PARA QUEM ESTÁ COMEÇANDO

Utilize este checklist para avaliar seu aprendizado.

CONCEITOS

- ☐ Sei explicar o que é programação.
- ☐ Sei explicar o que é um algoritmo.
- ☐ Consigo dividir um problema em etapas.

PYTHON

- ☐ Consigo utilizar print().
- ☐ Consigo utilizar input().
- ☐ Sei criar variáveis.
- ☐ Conheço os principais tipos de dados.
- ☐ Sei utilizar operadores.

LÓGICA

- ☐ Sei utilizar if, elif e else.
- ☐ Sei utilizar for.
- ☐ Sei utilizar while.
- ☐ Sei criar funções.
- ☐ Sei trabalhar com listas.

DESENVOLVIMENTO

- ☐ Consigo interpretar uma mensagem de erro.
- ☐ Sei testar meu código.
- ☐ Consigo identificar problemas simples de lógica.
- ☐ Sei organizar meu código.
- ☐ Consigo desenvolver um pequeno projeto.

PRÓXIMO PASSO

- desenvolvimento web
- desenvolvimento mobile
- banco de dados
- inteligência artificial
- análise de dados
- automação
- desenvolvimento de jogos
- engenharia de software

16. CONSIDERAÇÕES FINAIS

A programação é uma ferramenta utilizada para transformar problemas em soluções computacionais.

Durante este e-book, foram apresentados os principais fundamentos necessários para iniciar os estudos nessa área, começando por algoritmos e lógica de programação e avançando para variáveis, operadores, entrada e saída de dados, estruturas condicionais, repetições, funções e estruturas de dados.

A utilização de Python permitiu transformar os conceitos teóricos em exemplos práticos, tornando possível observar como uma ideia pode ser convertida em instruções executadas pelo computador.

Também foi apresentado um pequeno projeto de controle de gastos, demonstrando como diferentes conceitos podem ser combinados para desenvolver uma aplicação funcional.

É importante compreender que aprender programação exige prática. Não é necessário memorizar todos os comandos nem dominar uma linguagem imediatamente. O desenvolvimento acontece gradualmente por meio da resolução de problemas, experimentação, leitura de documentação e correção de erros.

O estudante que deseja continuar nessa área pode aprofundar seus conhecimentos em diferentes caminhos, como desenvolvimento de sistemas, aplicações web e mobile, inteligência artificial, análise de dados e engenharia de software.

Mais importante do que decorar comandos é desenvolver a capacidade de entender problemas, criar soluções, testar possibilidades e aprender continuamente.

17. REFERÊNCIAS E GLOSSÁRIO

REFERÊNCIAS

REFERÊNCIAS

- PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível na documentação oficial da linguagem Python.
- MOZILLA. MDN Web Docs. Documentação e materiais educativos para desenvolvimento web.
- MICROSOFT. Microsoft Learn: documentação para desenvolvedores. Documentação oficial.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. Porto Alegre: AMGH.
- WAZLAWICK, Raul Sidnei. Engenharia de Software: conceitos e práticas. Rio de Janeiro: Elsevier.

GLOSSÁRIO

- Algoritmo: sequência organizada de passos utilizada para solucionar um problema.
- Bug: comportamento incorreto ou problema em um programa.
- Código-fonte: conjunto de instruções escritas pelo programador.
- Debugging: processo de identificação e correção de erros.
- Função: bloco de código criado para executar uma tarefa específica.
- Linguagem de programação: linguagem utilizada para criar instruções que podem ser executadas por um computador.
- Lista: estrutura utilizada para armazenar vários elementos.
- Programa: conjunto de instruções executadas por um computador.
- Python: linguagem de programação utilizada em diversas áreas da tecnologia.
- Variável: espaço utilizado pelo programa para armazenar um valor.